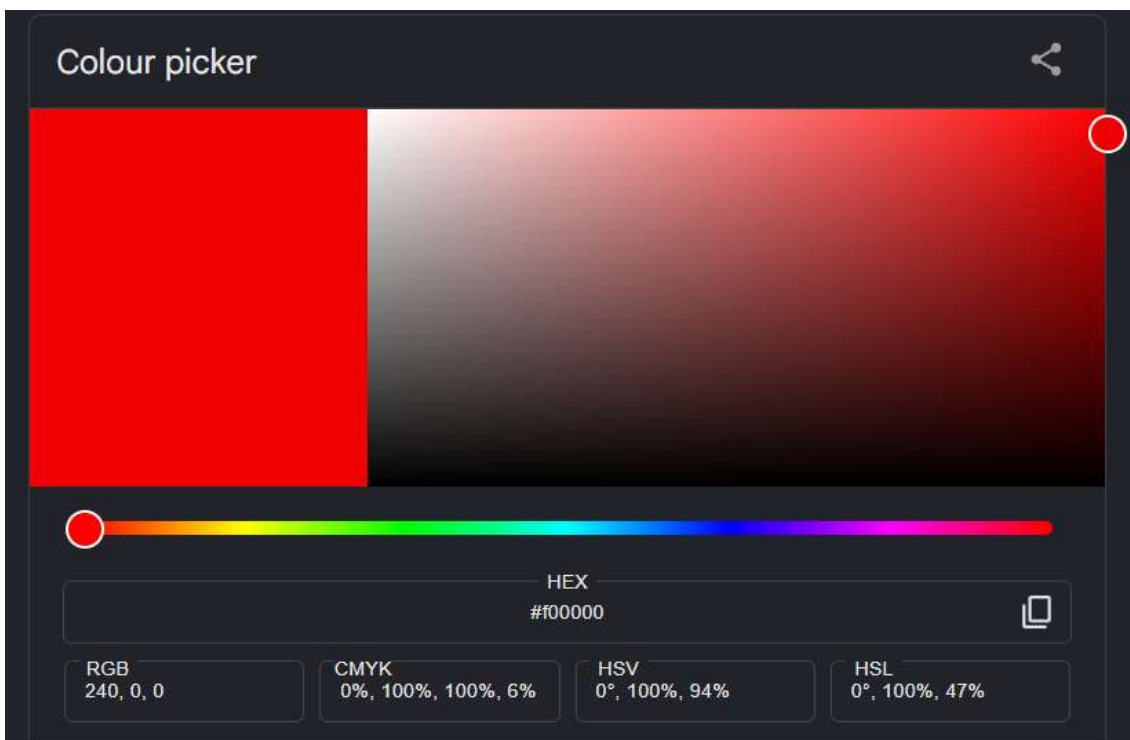
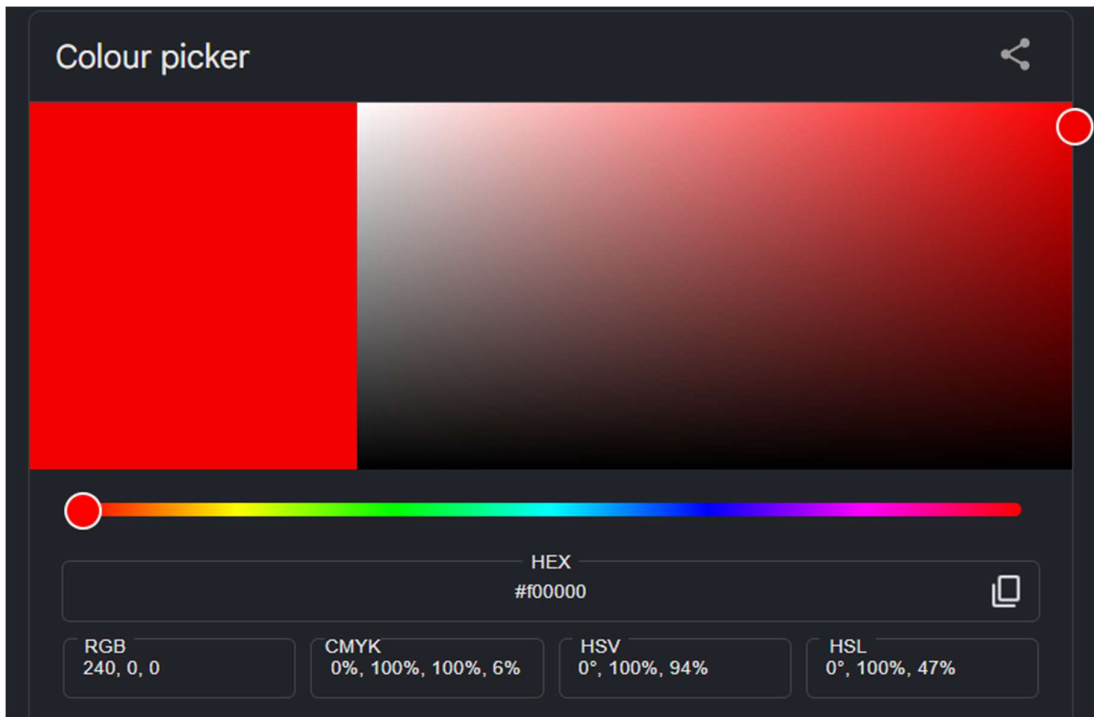


# ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

## SET 1 – C01 – PO1

### QUESTION 1 -



### Analysis of Hexadecimal Color Values:

| Color Code | Red Component (Hex) | Green Component (Hex) | Blue Component (Hex) | Red Binary Representation | Visual Output                     |
|------------|---------------------|-----------------------|----------------------|---------------------------|-----------------------------------|
| #FF0000    | FF                  | 00                    | 00                   | 11111111 <sub>2</sub>     | Max Intensity Pure Red            |
| #F00000    | F0                  | 00                    | 00                   | 11110000 <sub>2</sub>     | Slightly Darker, Less Intense Red |

### Supporting Explanation:

- **Binary Impact:** Changing the highest-order Red byte from FF (11111111<sub>2</sub>, or decimal 255) to F0 (11110000<sub>2</sub>, or decimal 240) reduces the voltage/intensity value sent to the monitor's red sub-pixels.
- **Result:** Because the red binary value decreased while green and blue remained at zero, the overall display output becomes a darker shade of red, creating the visual mismatch.

## QUESTION 2 –

### Case 1: Subtraction Resulting in Zero

The screenshot shows the GNUSim8085 - 8085 Microprocessor Simulator interface. The main window displays the assembly code and the state of the microprocessor registers and flags.

**Registers:**

| Register | Value |
|----------|-------|
| A        | 00    |
| BC       | 00 00 |
| DE       | 00 00 |
| HL       | 00 00 |
| PSW      | 00 00 |
| PC       | 42 04 |
| SP       | FF FF |
| Int-Reg  | 00    |

**Flags:**

| Flag | Value |
|------|-------|
| S    | 0     |
| Z    | 1     |
| AC   | 0     |
| P    | 1     |
| C    | 1     |

**Assembly Code:**

```
Load me at
1 MVI A, 05H ; Load 05H into Accumulator
2 SUB A      ; Subtract Accumulator from itself (05H - 05H)
3 HLT       ; Terminate program
```

**Memory:**

| Address (Hex) | Address | Data |
|---------------|---------|------|
| 0000          | 0       | 0    |
| 0001          | 1       | 0    |
| 0002          | 2       | 0    |
| 0003          | 3       | 0    |
| 0004          | 4       | 0    |
| 0005          | 5       | 0    |
| 0006          | 6       | 0    |
| 0007          | 7       | 0    |
| 0008          | 8       | 0    |
| 0009          | 9       | 0    |
| 000A          | 10      | 0    |
| 000B          | 11      | 0    |

**Assembler Message:**

```
Line No Assembler Message
0 Program assembled successfully
```

Simulator: Idle

### Case 2: Addition Causing Carry and Zero

The screenshot shows the GNUSim8085 - 8085 Microprocessor Simulator interface. The main window displays the assembly code and the state of the microprocessor registers and flags.

**Registers:**

| Register | Value |
|----------|-------|
| A        | 00    |
| BC       | 00 00 |
| DE       | 00 00 |
| HL       | 00 00 |
| PSW      | 00 00 |
| PC       | 42 04 |
| SP       | FF FF |
| Int-Reg  | 00    |

**Flags:**

| Flag | Value |
|------|-------|
| S    | 0     |
| Z    | 1     |
| AC   | 0     |
| P    | 1     |
| C    | 0     |

**Assembly Code:**

```
Load me at
1 MVI A, 80H ; Load 80H into Accumulator
2 ADD A      ; Add Accumulator to itself (80H + 80H)
3 HLT       ; Terminate program
```

**Memory:**

| Address (Hex) | Address | Data |
|---------------|---------|------|
| 0000          | 0       | 0    |
| 0001          | 1       | 0    |
| 0002          | 2       | 0    |
| 0003          | 3       | 0    |
| 0004          | 4       | 0    |
| 0005          | 5       | 0    |
| 0006          | 6       | 0    |
| 0007          | 7       | 0    |
| 0008          | 8       | 0    |
| 0009          | 9       | 0    |
| 000A          | 10      | 0    |
| 000B          | 11      | 0    |

**Assembler Message:**

```
Line No Assembler Message
0 Program assembled successfully
```

Simulator: Idle

### Case 1: Subtraction Resulting in Zero

- **Result in Accumulator:** 00H (00000000<sub>2</sub>)
- **Verified Flag Register Status:**
  - **Zero Flag (Z = 1):** Set to 1 because the result of the arithmetic subtraction (05H – 05H) is exactly zero.
  - **Parity Flag (P = 1):** Set to 1 because the binary result (00000000<sub>2</sub>) contains an even number of 1s (zero total bits, which counts as even parity).
  - **Sign Flag (S = 0) & Carry Flag (CY = 0):** Both remain reset to 0.

### Case 2: Addition Causing Carry and Zero

- **Result in Accumulator:** 00H (with an external carry generated)
- **Verified Flag Register Status:**
  - **Carry Flag (CY = 1):** Set to 1 because the binary addition (10000000<sub>2</sub> + 10000000<sub>2</sub> = 100000000<sub>2</sub>) overflows the 8-bit capacity of the accumulator, generating a carry-out bit.
  - **Zero Flag (Z = 1):** Set to 1 because the lower 8 bits remaining inside the accumulator register are exactly 00H.
  - **Parity Flag (P = 1):** Set to 1 because the 8 bits left in the accumulator (00000000<sub>2</sub>) have an even number of 1s.

## QUESTION 3 –

### Hardware Resource & Component Cooperation

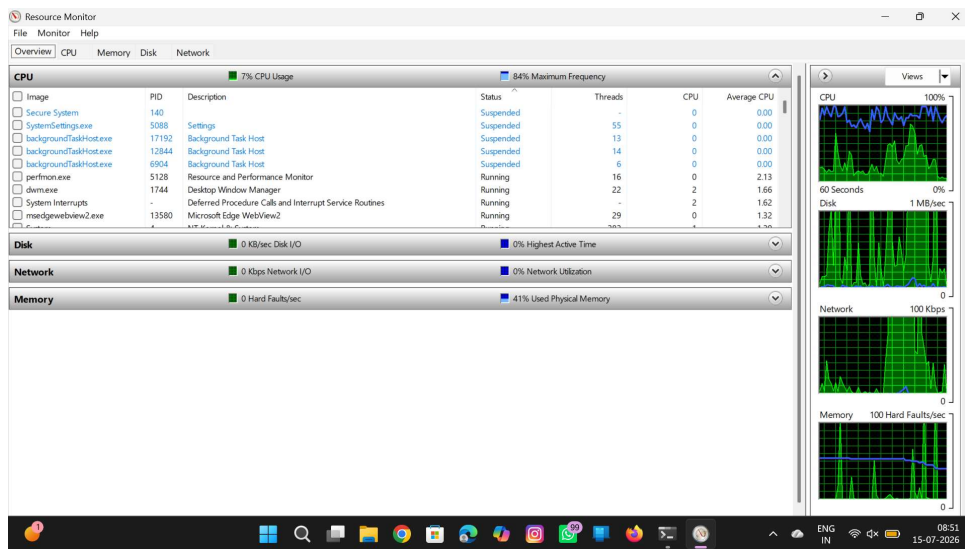
An operating system manages the execution of multiple applications simultaneously by coordinating the underlying hardware components. The interaction between these core components ensures seamless system operation:

- **CPU (Central Processing Unit):** Acts as the primary processing engine. It executes instructions and switches rapidly between threads (context switching) using scheduling algorithms. This creates the illusion of simultaneous execution for the document editor, web browser engine, and audio decoder.
- **Memory (RAM):** Serves as the high-speed temporary workspace. It holds the active instruction sets and data structures required by the operating system, the open browser tabs, the text buffers for the document editor, and the media streams for the audio player.
- **I/O Devices (Input/Output):** Handle communication between the user and the system.
  - *Input:* The keyboard and mouse trigger hardware interrupts that the CPU prioritizes to update the document layout in real time.
  - *Output:* The sound card receives decoded audio data from RAM and outputs analog signals to the speakers, while the Network Interface Card (NIC) continuously streams incoming web data packets.

### Multi-Tasking Resource Matrix

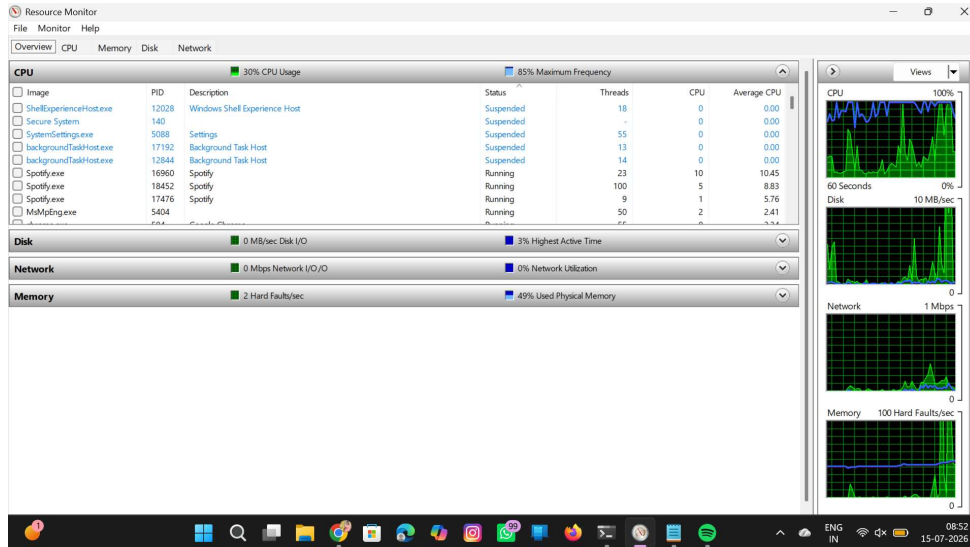
The following matrix outlines how system resources are distributed and utilized across these concurrent tasks:

| <b>Concurrent Task</b>   | <b>Primary Processing Unit</b>                                          | <b>Memory (RAM) Allocation</b>                                                         | <b>Target I/O Device</b>                                  |
|--------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <b>Editing Document</b>  | CPU (ALU and Control Unit handle text processing and layout formatting) | Caches the active text document, fonts, and undo/redo history buffers                  | Keyboard & Mouse (Input updates)                          |
| <b>Browsing Internet</b> | CPU (Processes JavaScript engines and renders DOM elements)             | Heavy memory footprints to hold assets, scripts, and separate processes per active tab | Network Interface Card / Wi-Fi (Data packet transmission) |
| <b>Playing Audio</b>     | CPU / Dedicated Digital Signal Processor (DSP) on the audio card        | Allocates small, continuous ring buffers to prevent audio stuttering or lagging        | Sound Card / Speakers (Analog audio output)               |



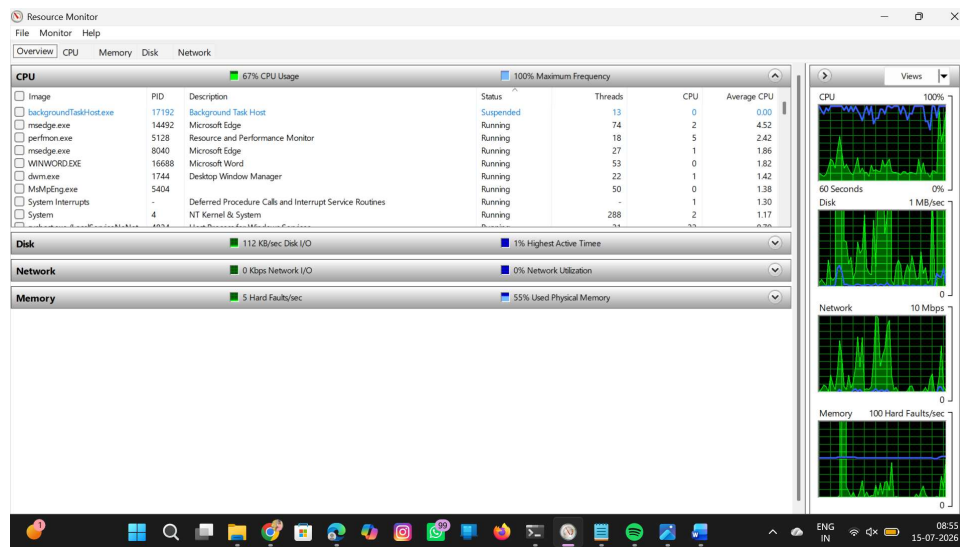
This is when the system is IDLE:

- The system is running with low workload, showing only 7% CPU usage and 41% memory usage. Most processes are idle or suspended, indicating that the computer is not performing any intensive tasks.



This is when AUDIO is playing in the background:

- While playing audio through Spotify, the CPU usage increases to 30% and memory usage rises to 49%. The CPU processes audio data, RAM stores the running application, and the speakers output the sound to the user.



During MULTITASKING (Word + Browser + Audio) in the system:

- When editing a document, browsing the internet, and playing audio simultaneously, CPU usage increases to 67% and memory usage reaches 55%. This shows how the CPU, memory, and I/O devices cooperate to handle multiple tasks efficiently, resulting in higher resource utilization.